

Use Case

- As a user who was released from a federal job, I want to provide my resume to the app and receive a list of private-sector jobs that are similar/equivalent to my old federal role
 - I can upload my resume and provide text input for additional information
 - The app can ask me relevant questions to elicit additional information if I didn't provide enough background or the app needs clarity on a role
 - The app will provide an explanation/metrics on why a role was a match
 - The output list should be short (no more than 10) so that I don't have a long list to wade through
 - The output should contain only roles with a high match to my previous role

(Soft) Technical specs

- Embed **O*NET**, **USAJobs.gov**, **resume**, and **private-sector data**
- Data on other coding systems (like MOC or OOH) will also be embedded.
- The app will use information like the user's previous job titles, descriptions, qualifications, SOC codes (or equivalents) to assist with the match
- Store the embeddings in Postgres/pgvector
- All embeddings derived from the same n-dimensional space
- Use cosine similarity to match jobs semantically
- Possibly run the model **locally** (no API needed)

Our focus

- **Domain relevance** (HR/talent/occupations)
- **Semantic embeddings** (not text generation)

✅ Embedding Models You Should Consider

Here's a comparison of models you might want to evaluate:

Model	Type	Params	Local?	API?	Strengths	Notes
all-MiniLM-L6-v2 (SBERT)	Embedding	~22M	✅ Yes	✅ (via HuggingFace)	Fast, good accuracy	Best all-around for low compute

Model	Type	Params	Local?	API?	Strengths	Notes
bge-small-en-v1.5	Embedding	~100M	✅ Yes	✅	Optimized for semantic search	Very high quality for size
bge-large-en-v1.5	Embedding	~1.2B	✅ (GPU)	✅	State-of-the-art search & retrieval	Best if using vector tuning
OpenAI text-embedding-3-small	Embedding (API)	Proprietary	❌	✅ (cheap)	Highly performant, optimized for search	Very strong out of the box

Other models that ChatGPT suggested:

Model	Type	Params	Local?	API?	Strengths	Notes
Cohere embed-english-v3.0	Embedding (API)	Proprietary	❌	✅	Strong performance, supports alignment	Generous free tier
Instructor-XL	Embedding	~560M	✅ (GPU)	✅	Uses task-specific instructions ("Represent this job description...")	Very flexible
GTE-Large	Embedding	~300M	✅	✅	Strong open model for semantic tasks	Works well with less tuning
Voyage AI	Embedding (API)	Proprietary	❌	✅	Known for retrieval quality	May outperform OpenAI in job tasks

👉 If You Want to Start Now (Offline + Solid Accuracy)

- ✅ **bge-small-en-v1.5** — high-quality embeddings, runs locally without GPU
- ✅ **all-MiniLM-L6-v2** — lightweight, fast, proven for job/HR data
- ✅ **GTE-Large** — another strong open-source alternative

If You Plan to Fine-Tune

- bge-large-en-v1.5 — excellent base for retrieval-augmented training
- Instructor-XL — allows you to guide the model with task instructions
- HuggingFace has adapters for **domain tuning with LoRA/QLoRA** (very memory efficient)

If You Want the Best Out-of-the-Box (API)

- OpenAI text-embedding-3-small – top-tier performance, ~\$0.0001/1K tokens
 - Cohere embed-english-v3.0 – great for cross-domain match
 - VoyageAI – slightly better than OpenAI for semantic job match use cases
-

Embedding vs LLMs

Just to clarify:

- You're not using a full LLM for generation — you're using **encoder models** (BERT-style or Transformer-style) to convert job/resume text into dense vectors.
 - These models **don't need to be OpenAI/GPT-sized** — and often run on CPU.
-

Final Suggestion

Start with:

- bge-small-en-v1.5 or all-MiniLM-L6-v2 (local)
- Try text-embedding-3-small (API) for comparison
- Later: Fine-tune bge-large on your own labeled pairs